ETC3250/5250 Introduction to Machine Learning

Week 3: Re-sampling

Jack Jewson

Jack.Jewson@monash.edu

Department of Econometrics and Business Statistics

ETC3250/5250 Lecture 3

Overview

We will cover:

- Common re-sampling methods: **bootstrap**, **cross-validation**, **permutation**, **simulation**.
- **Cross-validation** for checking generalisability of model fit, parameter tuning, variable selection.
- **Bootstrapping** for understanding variance of parameter estimates.
- **Permutation** to understand significance of associations between variables, and variable importance.
- **Simulation** can be used to assess what might happen with samples from known distributions.

Model development and choice

ETC3250/5250 Lecture 3

How do you get new data?

ETC3250/5250 Lecture 3

Re-sampling methods

- **Cross-validation**: Splitting the data into multiple samples.
- **Bootstrap**: Sampling with replacement
- **Permutation**: Re-order the values of one or more variables

- Cross-validation: This is used to gain some understanding of the out-of-sample performance of models, and how parameter or algorithm choices affect this performance.
  - Bootstrap: Compute confidence intervals for model parameters, or the model fit statistics. Can be used similarly to cross-validation samples but avoids the complication of smaller sample size that may affect interpretation of cross-validation samples.

- Permutation: Used to assess significance of relationships, especially to assess the importance of individual variables or combinations of variables for a fitted model.

ETC3250/5250 Lecture 3

Cross-validation

- Training/test split: make one split of your data, keeping one purely for assessing future performance. **ALWAYS DO THIS**

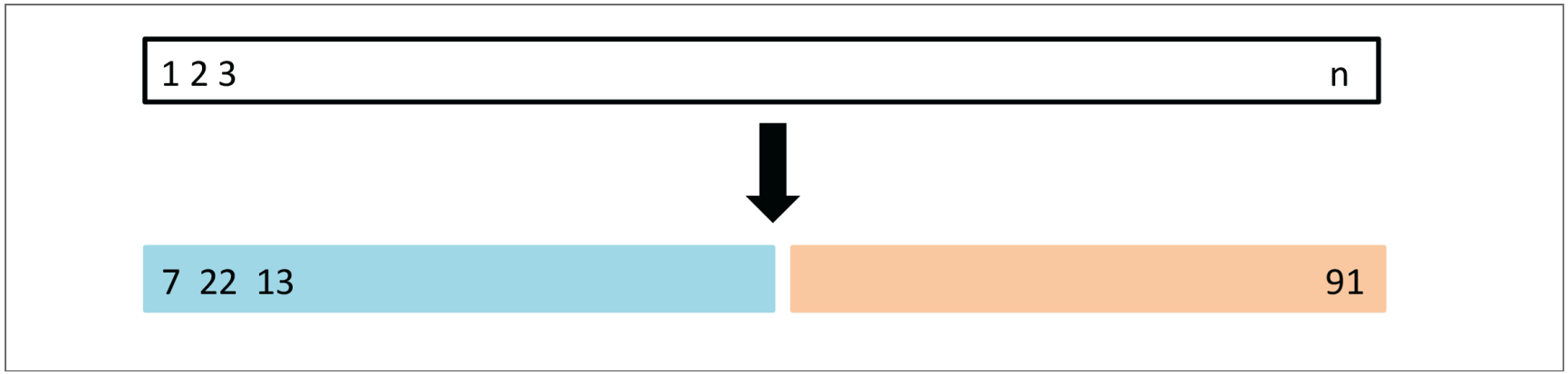
After making that split, we would use these methods on the training sample:

- Leave-one-out: make n splits, fitting multiple models and using left-out observation for assessing variability.
- k-fold: break data into k subsets, fitting multiple models with one group left out each time.

If we use the test set to make model selection decision like learn hyperparameters, then we have trained using the test set, and it is **no longer** a test set
Cross-validation makes mini training/testing splits within our training set

ETC3250/5250 Lecture 3

Training/test split (1/3)



A set of n observations are randomly split into a training set (blue, containing observations 7, 22, 13, …) and a test set (yellow, all other observations not in training set).

(Chapter5/5.1.pdf)

- Need to stratify the sampling to ensure training and test groups are appropriately balanced.
- Make sure the observations are not kept in order

ETC3250/5250 Lecture 3

- Only one split of data made, may have a lucky or unlucky split, but accurately estimating test error relies on the one sample.

Training/test split (2/3)

With tidymodels, the function `initial_split()` creates the indexes of observations to be allocated into training or test samples. To generate these samples use `training()` and `test()` functions.

```
d_bal <- tibble(y=c(rep("A", 6), rep("B", 6)),
                x=c(runif(12)))
d_bal$y
```

```
[1] "A" "A" "A" "A" "A" "A" "B" "B" "B" "B" "B" "B"
```

```
set.seed(130)
d_bal_split <- initial_split(d_bal, prop = 0.70)
training(d_bal_split)$y
```

```
[1] "A" "A" "B" "A" "B" "A" "B" "A"
```

```
testing(d_bal_split)$y
```

```
[1] "A" "B" "B" "B"
```

How can you ensure that the class proportions are maintained in both the training and testing sets?

Stratify the sampling

```r
d_bal$y
```

```
[1] "A" "A" "A" "A" "A" "A" "B" "B" "B" "B" "B" "B"
```

```r
set.seed(1225)
d_bal_split <- initial_split(d_bal,
                             prop = 0.70,
                             strata=y)
training(d_bal_split)$y
```

```
[1] "A" "A" "A" "A" "B" "B" "B" "B"
```

```r
testing(d_bal_split)$y
```

```
[1] "A" "A" "B" "B"
```

ETC3250/5250 Lecture 3

Now the test set has 2 A's and 2 B'2. This is best practice!

ETC3250/5250 Lecture 3

Training/test split (3/3)

Not stratifying can cause major problems with unbalanced samples.

```
d_unb <- tibble(y=c(rep("A", 2), rep("B", 10)),
                x=c(runif(12)))
d_unb$y
```

```
[1] "A" "A" "B" "B" "B" "B" "B" "B" "B" "B" "B" "B"
```

```
set.seed(132)
d_unb_split <- initial_split(d_unb, prop = 0.70)
training(d_unb_split)$y
```

```
[1] "B" "B" "A" "B" "B" "A" "B" "B"
```

```
testing(d_unb_split)$y
```

```
[1] "B" "B" "B" "B"
```

The test set is missing one entire class!

Always stratify splitting by sub-groups, especially response variable classes, and possibly other variables too.

```
d_unb_strata <- initial_split(d_unb,
                             prop = 0.70,
                             strata=y)
training(d_unb_strata)$y
```

```
[1] "A" "B" "B" "B" "B" "B" "B" "B"
```

```
testing(d_unb_strata)$y
```

```
[1] "A" "B" "B" "B"
```

Now there is an A in the test set!

ETC3250/5250 Lecture 3

Unbalanced classification

- In binary classification, the classes are referred to as unbalanced if there are many more Class A's than Class B's
- This happens in most interesting applications where you want to detect something rare
- Consider for example 90% of the observations are Class A and 10% are Class B.
- It becomes hard to diagnose the model's performance
- For example, predicting everything is class A with probability 1 will achieve accuracy 0.9, and a pretty high AUC

Unbalanced classification

Have to look at the whole Confusion matrix

		predicted	
		1	0
true	1	a	b
	0	c	d

- Accuracy = (a+d)/(a+b+c+d)
- Balanced accuracy = a/(2(a+b)) + d/(2(c+d))
- in the 90:10 case predicting everything as Class A
  - $a = 90$, $c = 10$, $b = 0$ and $d = 0$
  - Accuracy is 0.9

ETC3250/5250 Lecture 3

- Balanced accuracy is 1/2 + 0/2 = 0.5

ETC3250/5250 Lecture 3

Checking the training/test split: response

Check the class proportions of the response by computing counts and proportions in each class, and tabulating or plotting the result.

It's good if there are similar numbers of each class in both sets.

GOOD

BAD

ETC3250/5250 Lecture 3

Checking the training/test split: predictors

Make a training/test variable and plot the predictors. Need to have similar distributions.

GOOD

Looks good

On the response training and test sets have similar proportions of each class so looks good BUT it's not

But BAD

ETC3250/5250 Lecture 3

Test set has smaller penguins on at least two of the variables.

ETC3250/5250 Lecture 3

Cross-validation

ETC3250/5250 Lecture 3

k-fold cross validation (1/4)

1. Divide the data set into k different parts.
2. Remove one part, fit the model on the remaining $k-1$ parts, and compute the **statistic of interest** on the omitted part.
3. Repeat k times taking out a different part each time

k-fold cross validation (2/4)

1. Divide the data set into k different parts.
2. Remove one part, fit the model on the remaining $k-1$ parts, and compute the **statistic of interest** on the omitted part.
3. Repeat k times taking out a different part each time

Here are the row numbers for $k = 5$ folds:

```
p_folds <- vfold_cv(p_sub, 5, strata=species)
c(1:nrow(p_sub))[-p_folds$splits[[1]]$in_id]
```

```
[1]  5  6  8 12 16 23 28 31 43 44 45 53 57 58 70 73 74 77
```

```
c(1:nrow(p_sub))[-p_folds$splits[[2]]$in_id]
```

```
[1]  2  9 10 11 13 17 22 25 39 48 50 51 55 61 65 69 75 78
```

```
c(1:nrow(p_sub))[-p_folds$splits[[3]]$in_id]
```

ETC3250/5250 Lecture 3

```
[1]  1  3 14 18 20 26 33 41 42 49 56 67 72 81 83 84
```

```r
c(1:nrow(p_sub))[-p_folds$splits[[4]]$in_id]
```

```
[1]  4 19 29 32 34 35 36 40 46 52 63 64 66 76 79 80
```

```r
c(1:nrow(p_sub))[-p_folds$splits[[5]]$in_id]
```

```
[1]  7 15 21 24 27 30 37 38 47 54 59 60 62 68 71 82
```

ETC3250/5250 Lecture 3

k-fold cross validation (3/4)

1. Divide the data set into k different parts.
2. Remove one part, fit the model on the remaining $k-1$ parts, and compute the **statistic of interest** on the omitted part.
3. Repeat k times taking out a different part each time

Fit the model to the $k-1$ set, and compute the statistic on the k-fold, that was not used in the model fit.
Here we use the **accuracy** as the statistic of interest.
Value for fold 1 is:

```
# A tibble: 1 × 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy multiclass     0.889
```

ETC3250/5250 Lecture 3

k-fold cross validation (4/4)

1. Divide the data set into k different parts.
2. Remove one part, fit the model on the remaining $k-1$ parts, and compute the **statistic of interest** on the omitted part.
3. Repeat k times taking out a different part each time

Here is the **accuracy** computed for each of the $k = 5$ folds. Remember, this means that the model was fitted to the rest of the data, and accuracy was calculate on the observations in this fold.

```
[1] 0.89 0.89 1.00 0.88 1.00
```

Recommended reading: Alison Hill's **Take a Sad Script & Make it Better: Tidymodels Edition**

k-fold cross validation uses

- Cross-validation gives you an idea of how a particular model performs out-of-sample
- Use this to select model hyperparameters e.g. LASSO regularisation parameter, depth of decision tree or Neural Network architecture
- Evaluate cross-validated accuracy for different hyperparameters and proceed with the hyperparameter with the ‘best’ out of sample performance

- Cross-validation allows us to average over different test/train splits so provides a more reliable measure of model performance
- However, we have still ‘trained’ on the cross-validated test sets (as we used those to learn hyperparameters) hence we should still evaluate our model using the test split
- **Note** your final fitted model should be re-fit on ALL of the training data

ETC3250/5250 Lecture 3

LOOCV

> Leave-one-out (LOOCV) is a special case of k-fold cross-validation, where $k = n$. There are n CV sets, each with ONE observation left out.

Benefits:

- Useful when sample size is very small.
- Some statistics can be calculated algebraically, without having to do computation for each fold.
- Gives you the best measure of performance

But:

- Model needs to be fit $k = n$ times and n will be very large

Where is cross-validation used?

- Model evaluation and selection, by estimating the generalisability on future data.
- Parameter tuning: finding optimal choice of parameters or control variables, like number of trees or branches, or polynomial terms to generate the best model fit.
- Variable selection: which variables are more or less important for the best model fit. Possibly some variables can be dropped from the model without impacting the corss-validated error.

Cross validation to select K in kNN (1/5)

- Specify a distance metric $d(\cdot, \cdot)$ (usually defaults to $d(x_1, x_2) = \sqrt{\sum_{j=1}^{p}(x_{1j} - x_{2j})^2}$) and a number of neighbours K
- When predicting new observation x'
  - for $i = 1, \ldots, n$ evaluate $d(x', x_i)$
  - select the K observations with the smallest $d(x', x_i)$
  - Predict the response as the average of the y_i of the K nearest neighbours
    - In regression predict $\frac{1}{K}\sum_{k=1}^{K} y_{n_k}$
    - In classification predict $P(y = C) = \frac{1}{K}\sum_{k=1}^{K} \mathbb{I}(y_{n_k} = C)$

ETC3250/5250 Lecture 3

Cross validation to select K in kNN (2/5)

`tidymodels` implementation (**https://parsnip.tidymodels.org/reference/nearest_neighbor.html**)

```
K <- 5
```

```
knn_spec <- nearest_neighbor(neighbors = K, weight_func = "rectangu
  set_mode("classification") |>
  set_engine("kknn") #specify how to learn the parameters
```

```
knn_fit <- knn_spec |> ## apply the data
  fit(y ~ ., data = sim_data2)
```

ETC3250/5250 Lecture 3

ETC3250/5250 Lecture 3

Cross validation to select K in kNN (3/5)

Cross-validation in the `tidymodels`

```
tune_spec <-
  nearest_neighbor(
    neighbors = tune(),
    weight_func = "rectangular",
    dist_power = 2
  ) |>
  set_mode("classification") |>
  set_engine("kknn") #specify how to learn the parameters
```

```
neighbour_grid <- grid_regular(neighbors(),
                               levels = 5)
```

```
neighbour_grid$neighbors
```

ETC3250/5250 Lecture 3

```
[1]  1  3  5  7 10
```

```r
set.seed(234)
p_folds <- vfold_cv(sim_data2, v = 5, strata = y)
```

```r
kNN_wf <- workflow() |>
  add_model(tune_spec) |>
  add_formula(y~.)
```

```r
set.seed(345)
kNN_res <-
  kNN_wf |>
  tune_grid(
    resamples = p_folds,
    grid = neighbour_grid,
    metrics = NULL #NULL to compute a standard set of metrics.
    )
```

```r
kNN_res |>
  collect_metrics() |>
```

ETC3250/5250 Lecture 3

```
  slice_head(n = 6)
```

```
# A tibble: 6 × 7
  neighbors .metric  .estimator   mean     n std_err .config
      <int> <chr>    <chr>       <dbl> <int>   <dbl> <chr>
1         1 accuracy binary     0.84       5  0.0384 Prepro…
2         1 brier_c… binary     0.16       5  0.0384 Prepro…
3         1 roc_auc  binary     0.773      5  0.0446 Prepro…
4         3 accuracy binary     0.885      5  0.0232 Prepro…
5         3 brier_c… binary     0.0944     5  0.0151 Prepro…
6         3 roc_auc  binary     0.906      5  0.0233 Prepro…
```

ETC3250/5250 Lecture 3

Cross validation to select K in kNN (4/5)

```
kNN_res_roc <- kNN_res |>
  collect_metrics() |>
  filter(.metric == "roc_auc") |>
  select(neighbors, mean, std_err)

kNN_res_roc |>
ggplot(aes(x = neighbors, y = mean)) +
   geom_line() + # Use geom_col for pre-computed heights
   geom_errorbar(aes(ymin = mean - 2*std_err, ymax = mean + 2*std_e
```

Looking to see the criteria (roc-auc) maximised in the interior of the search region

If it happens on the upper or lower boundary, then you need to shift the region higher or lower

View this as an iterative procedure

- Start with a wide sparse grid

ETC3250/5250 Lecture 3

- Once you obtain a solution in the interior, zoom in on that region

ETC3250/5250 Lecture 3

Cross validation to select K in kNN (5/5)

```
kNN_res |>
  show_best(metric = "roc_auc")
```

```
# A tibble: 5 × 7
  neighbors .metric .estimator  mean     n std_err .config
      <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
1         7 roc_auc binary     0.94      5 0.00932 Preproce…
2        10 roc_auc binary     0.936     5 0.0165  Preproce…
3         5 roc_auc binary     0.928     5 0.0122  Preproce…
4         3 roc_auc binary     0.906     5 0.0233  Preproce…
5         1 roc_auc binary     0.773     5 0.0446  Preproce…
```

```
kNN_res |>
  select_best(metric = "roc_auc")
```

```
# A tibble: 1 × 2
  neighbors .config
```

ETC3250/5250 Lecture 3

```
       <int> <chr>
1          7 Preprocessor1_Model4
```

For more information see - https://www.tidymodels.org/start/tuning/

ETC3250/5250 Lecture 3

Bootstrap

ETC3250/5250 Lecture 3

Bootstrap (1/7)

- A bootstrap sample is a sample that is the same size as the original data set
- Made by sampling n times from the observations with replacement.
- Imagine a bag with balls with numbers $1, \ldots, n$ written on them
  - I pick a ball
  - Write down its number
  - and replace the ball, before picking again
- This results in samples that have multiple replicates of some of the original rows of the data.
- The assessment set is defined as the rows of the original data that were not included in the bootstrap sample, referred to as the **out-of-bag** (OOB) sample.

Bootstrap (2/7)

```r
set.seed(115)
df <- tibble(id = 1:26,
             cl = c(rep("A", 12), rep("B", 14)))
df_b <- bootstraps(df, times = 100, strata = cl)
```

```r
t(df_b$splits[[1]]$data[df_b$splits[[1]]$in_id,])
```

```
   [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11]
id " 1" " 2" " 2" " 2" " 2" " 5" " 6" " 7" " 9" "11"  "12"
cl "A"  "A"  "A"  "A"  "A"  "A"  "A"  "A"  "A"  "A"   "A"
   [,12] [,13] [,14] [,15] [,16] [,17] [,18] [,19] [,20]
id "12"  "14"  "14"  "18"  "18"  "18"  "18"  "18"  "19"
cl "A"   "B"   "B"   "B"   "B"   "B"   "B"   "B"   "B"
   [,21] [,22] [,23] [,24] [,25] [,26]
id "21"  "21"  "21"  "22"  "25"  "25"
cl "B"   "B"   "B"   "B"   "B"   "B"
```

Which observations are out-of-bag in bootstrap sample 1?

ETC3250/5250 Lecture 3

Bootstrap (3/7)

- Bootstrap is preferable to cross-validation when the sample size is small, or if the structure in the data being modelled is complex.
- It is commonly used for estimating the variance of parameter estimates, especially when the data is non-normal.
- We calculate the parameter estimate on each bootstrap sample and this gives us an estimate of the distribution for the estimate

Bootstrap (4/6)

- In dimension reduction it can be used to assess if the coefficients of a PC (the eigenvectors) are significantly different from ZERO.
- The 95% bootstrap confidence intervals can be computed by:

1. Generating B bootstrap samples of the data
2. Compute PCA, record the loadings
3. Re-orient the loadings, by choosing one variable with large coefficient to be the direction base (the axes are the same whether they are pointing forwards or backwards)
4. If B=1000, 25th and 975th sorted values yields the lower and upper bounds for confidence interval for each PC.

Bootstrap (5/7)

Assessing the loadings for PC 2 of PCA on the womens track data. Remember the summary:

```
Standard deviations (1, .., p=7):
[1] 2.41 0.81 0.55 0.35 0.23 0.20 0.15

Rotation (n x k) = (7 x 7):
          PC1   PC2    PC3    PC4    PC5     PC6    PC7
m100     0.37  0.49 -0.286  0.319  0.231  0.6198  0.052
m200     0.37  0.54 -0.230 -0.083  0.041 -0.7108 -0.109
m400     0.38  0.25  0.515 -0.347 -0.572  0.1909  0.208
m800     0.38 -0.16  0.585 -0.042  0.620 -0.0191 -0.315
m1500    0.39 -0.36  0.013  0.430  0.030 -0.2312  0.693
m3000    0.39 -0.35 -0.153  0.363 -0.463  0.0093 -0.598
marathon 0.37 -0.37 -0.484 -0.672  0.131  0.1423  0.070
```

Should we consider `m800`, `m400` contributing to PC2 or not?

ETC3250/5250 Lecture 3

i.e. are ϕ_{32} and ϕ_{42} are *significantly* different from 0?

ETC3250/5250 Lecture 3

Bootstrap (6/7)

We said that *PC2 is a contrast between short distance events and long distance events, particularly 100m, 200m vs 1500m, 3000m, marathon*.

How reliably can we state this?

Note that the data was standardised once before bootstrapping

- Code

Bootstrap (7/7)

Confidence intervals for `m400` and `m800` cross ZERO, hence zero is a plausible value for the population coefficient corresponding to this estimate.

Under at least some the boostrap re-samples of the data the loadings of `m400` and `m800` on PC2 changes sign and/or is very close to 0

Permutation

ETC3250/5250 Lecture 3

Permutation (1/4)

- A permutation is a random reordering
- One permutation of $(1, 2, 3, 4, 5)$ is $(2, 5, 4, 3, 1)$
- While bootstrap sampled with replacement, a permutation samples without replacement
- Here we permute some of the columns of our data
- breaks relationships between variables
- often used for conducting statistical hypothesis tests, without requiring too many assumptions

Permutation (2/4)

DATA

```
# A tibble: 10 × 2
       x cl
   <dbl> <chr>
 1 0.281 A
 2 0.330 A
 3 0.708 A
 4 0.463 A
 5 3.37  A
 6 0.528 B
 7 0.852 B
 8 5.58  B
 9 0.685 B
10 3.28  B
```

PERMUTE `cl`

```
# A tibble: 10 × 2
       x cl
   <dbl> <chr>
 1 0.281 A
 2 0.330 B
```

ETC3250/5250 Lecture 3

```
 3 0.708 A
 4 0.463 B
 5 3.37  B
 6 0.528 A
 7 0.852 B
 8 5.58  B
 9 0.685 A
10 3.28  A
```

If there is a relationship between two variables, then this relationship should be destroyed by permutation

Is there a difference in the medians of the groups?

If a relationship is similar after permutation, then it must have been purely due to chance

ETC3250/5250 Lecture 3

Permutation (3/4)

Is there a difference in the medians of the groups?

Similarly with bootstrap, we can now assess the distribution of this dependence with multiple permutations

Generate k permutation samples, compute the medians for each, and compare the difference with original.

- Code

Permutation (4/4)

- Caution: permuting small numbers, especially classes may return very similar samples to the original data.
- Stay tuned for random forest models, where permutation is used to help assess the importance of all the variables.

- Can think about fitting the model, and comparing the model fit to a fit on permuted data
- More often used to screen whether variables might be important or not to model a response

Comparison

- Cross-validation: creates *smaller* training and testing sets to evaluate model's out-of-sample performance
- Bootstrap: creates training data of the same size to understand the distribution of training parameter estimates
  - Samples whole observation rows
  - Samples with replacement
- Permutation: creates training data of the same size to understand relationships between variables (without a model)
  - Samples subsets of rows
  - Samples without replacement

Simulation

ETC3250/5250 Lecture 3

Simulation (1/6)

Simulating from known statistical distributions allows us to check data and calculations against what is known in controlled conditions.

We know the truth, and so we know how we want our method or model to behave?

For example, how likely is it to see the extreme a value if my data is a sample from a normal distribution?

As we're simulating data we can simulate as much as we like

Simulation (2/6)

Consider doing PCA with $p = 2$ dimensional data with $x_1 \sim \mathcal{N}(0, 1^2)$ and $x_2 = x_1 + \mathcal{N}(0, 0.1^2)$

- What should the PCA look like?
- What will the proportion of variances be?
- How should the PC loadings look

Simulation (3/6)

Consider doing PCA with $p = 2$ dimensional data with $x_1 \sim \mathcal{N}(0, 1^2)$ and $x_2 = x_1 + \mathcal{N}(0, 0.1^2)$

```r
data_pca <- prcomp(data, retx=T, center=T, scale=TRUE)

data_pca$sdev
```

```
[1] 1.41 0.07
```

```r
data_pca$rotation
```

```
     PC1   PC2
x1 -0.71 -0.71
x2 -0.71  0.71
```

ETC3250/5250 Lecture 3

Simulation (4/6)

What about UMAP

```
library(uwot)
set.seed(253)
data_umap <- umap(data, init = "spca")

data_umap_df <- tibble(x1 = data_umap[,1],
                       x2 = data_umap[,2])
```

Simulation (5/6)

What about UMAP

Simulation (6/6)

```
mulgar::ggscree(track_pca, q=7) +
  ggtitle("Scree plot of womens track data")
```

- Grey line is a guide line, computed by doing PCA on 100 samples from a standard p-dimensional normal distribution (with no dependence).
- That is a comparison of the correlation matrix of the track data with a correlation matrix that is the identity matrix, where there is no association between variables.
- The largest variance we expect is under 2. The observed variance for PC 1 is much higher. Much larger than expected, very important for capturing the variability in the data!
- Why is there a difference in variance, when there is no difference in variance?

Next: Logistic regression and discriminant analysis

ETC3250/5250 Lecture 3